HW 1 – Datasets and Preprocessing Data

MMAI 2026 – Multimodal Machine Learning

Peiran Niu

February 2026

Part 1: Project Objective & Data Planning (25 pts)

1.1 Model Goals

My project, S-MIP-Verifier, is a multimodal system for cross-modal MILP verification. A Mixed-Integer Linear Program (MILP) is an optimization problem where some variables must be integers and all objectives/constraints are linear. Given a natural-language (NL) business specification and a MILP formulation, the system checks whether the two are consistent and, when they are not, diagnoses the specific error (e.g., flipped inequality, wrong variable type, missing constraint). This addresses a real need in Operations Research: a MILP can solve without error yet produce wrong results if the formulation does not match the intended logic.

A secondary goal is archetype classification: predicting which problem class a MILP belongs to (Set Cover, Knapsack, Facility Location, etc.) to inform solver configuration.

1.2 Datasets

I identified three datasets relevant to the project:

1. Distributional MIPLIB (Huang et al., 2024) — https://sites.google.com/usc.edu/distributional-miplib/home
A standardized benchmark of MILP instances across 13 domains (Set Cover, Facility Location, Combinatorial Auction, etc.), classified by difficulty. Drawback: no natural-language descriptions, so it cannot support text-structure alignment on its own.

2. MILP-Evolve (Li et al., 2025) — https://huggingface.co/datasets/microsoft/MILP-Evolve
Paired (NL description, MILP instance) data generated by an LLM-based framework — essential for cross-modal verification. Drawback: descriptions are templated and the dataset is large.

3. NLP4LP — https://nlp4lp.vercel.app/
Human-written optimization problem descriptions (ComplexOR and NL4OPT subsets). Drawback: no paired structured MILP files.

For HW1, I use Distributional MIPLIB as the reference taxonomy to guide my synthetic generators, since standard parsers (SCIP, python-mip) require C++ compilers unavailable on Colab. MILP-Evolve and NLP4LP will be used in later project assignments.

1.3 Modality Selection

I extract three modalities from each MILP instance:

Modality 1 — Constraint Matrix (Numerical): coefficient matrix A, RHS vector b, objective vector c, variable bounds and types. Preserves full numerical detail for detecting coefficient-level errors.

Modality 2 — Bipartite Graph (Topological): constraint nodes connected to variable nodes via edges weighted by non-zero entries in A, with node attributes (constraint sense, variable type). Captures connectivity patterns suited for GNN encoders and surfaces structure invisible in a flat matrix.

Modality 3 — Symbolic Attribute Vector (Statistical Summary): a 22-dim feature vector of problem size, matrix density, variable type fractions, constraint sense fractions, graph degree statistics, and objective/RHS summary statistics. A compact fingerprint for quick domain classification.

Excluded: solver runtime features (require solving the MILP, contradicting pre-solve verification); natural-language descriptions (not available for HW1); matrix heatmap images (lossy and redundant given the numerical modality).

1.4 Data Acquisition Difficulties

1. Solver dependencies: SCIP, Gurobi, and python-mip all require C++ compilers or commercial licenses unavailable on Colab. I worked around this by writing pure-Python generators that produce sparse constraint matrices directly.

2. Scale heterogeneity: Generated instances vary enormously — Knapsack averages ~5 constraints and ~81 variables, while Bin Packing averages ~893 variables. I standardized all features (zero mean, unit variance) before t-SNE to prevent size from dominating.

3. Limited intra-class diversity: Fixed generator templates produce structurally similar instances within each domain, unlike real MIPLIB files. This is acceptable for HW1 but will be addressed by switching to real data in later assignments.

4. No text modality: Synthetic instances have no NL annotations. Cross-modal alignment must wait until MILP-Evolve is introduced.

5. One-to-many text-structure mapping (anticipated): A single sentence like 'every customer must be served by exactly one facility' can map to hundreds of matrix rows — a key alignment challenge for later project phases.

1.5 Addressing the Six Core Challenges

Below we discuss how each of the six core challenges of multimodal learning (Liang et al., 2022) applies to our dataset design.

1.5.1 Representation

My three modalities are fundamentally different in format: a sparse 2D numerical matrix, a discrete bipartite graph, and a fixed-length real-valued vector. I plan to use modality-specific encoders (sparse matrix encoder, GNN, MLP) each producing a fixed-dimensional embedding,

fused via concatenation or cross-attention. For HW1, each modality is stored in its native format to preserve structure before downstream fusion.

1.5.2 Alignment

At the instance level, alignment is trivial: each instance yields one matrix, one graph, and one attribute vector linked by instance ID. Row/column indices are preserved across modalities for element-level traceability. When text is introduced via MILP-Evolve, alignment becomes harder — a phrase like 'at most one item per bin' must link to specific <= constraints, involving ambiguous segmentation and one-to-many mappings.

1.5.3 Reasoning

Verification needs reasoning to check if the math matches the text. The model will need to extract attributes from both sides and compare them.

1.5.4 Generation

Generation is relevant for creating training data. I will generate 'bad' examples by flipping signs or changing variable types to train the model to spot errors.

1.5.5 Transference

Since text data is scarce, I plan to use a pre-trained language model to transfer knowledge, which should help performance.

1.5.6 Quantification

I quantify multimodal learning mainly by looking at cross-modality feature correlations, like how graph statistics relate to the matrix density. I also used t-SNE to see if the domains separate well. This helps understand the data distribution.

Part 2: Multimodal Data Extraction (20 pts)

2.1 Extracted Modalities

I generated 180 synthetic MILP instances (30 per domain) using Python scripts because installing real solvers on Colab was failing.

Modality 1 — Constraint Matrix: stored in CSR sparse format along with the vectors.

Modality 2 — Bipartite Graph: built using NetworkX. Constraints and variables are nodes, and non-zero coefficients are edges. It helps visualize the structure.

Modality 3 — Symbolic Attribute Vector: A vector containing stats like number of variables, constraints, density, and fraction of binary variables.

2.2 Difficulties Encountered

The main difficulty was that python-mip failed to install on Colab, probably due to some missing system libraries. I had to write custom generators instead which took time.

Also, the instance sizes vary a lot. Knapsack is small but dense, while others are large and sparse. This made the raw features look weird until I normalized them.

NetworkX is also kind of slow for the larger graphs, but it worked okay for this small dataset.

Part 3: Data Visualization (15 pts)

3.1 Visualization Methods & Discussion

I visualized the data using t-SNE and some other plots.

— Data Distribution —

Figure 1 (t-SNE by Domain): The 6 domains separate into clusters pretty well. Knapsack is very distinct because it's fully dense. Set Cover and Combinatorial Auction overlap a bit because they are similar problems.

Figure 2 (Perplexity): I tried different perplexity values. 15 seemed to look the best, giving a good balance between local and global structure.

— Samples —

Figure 3 (Sparsity): Spy plots show the matrix structure. Knapsack is a block, while others are more diagonal.

Figures 8–9 (Graph Layouts): I generated small versions of the graphs to visualize them because the full ones are too messy. You can see the bipartite structure clearly.

— Input Distribution —

Figure 4 (Feature Distributions): Density is a key feature. Knapsack is 1.0, others are low.

Figure 7 (Correlation): Shows that some features like binary fractions and continuous fractions are opposites. Also, the number of variables correlates with the number of non-zeros, as expected.

3.2 Visualization Results

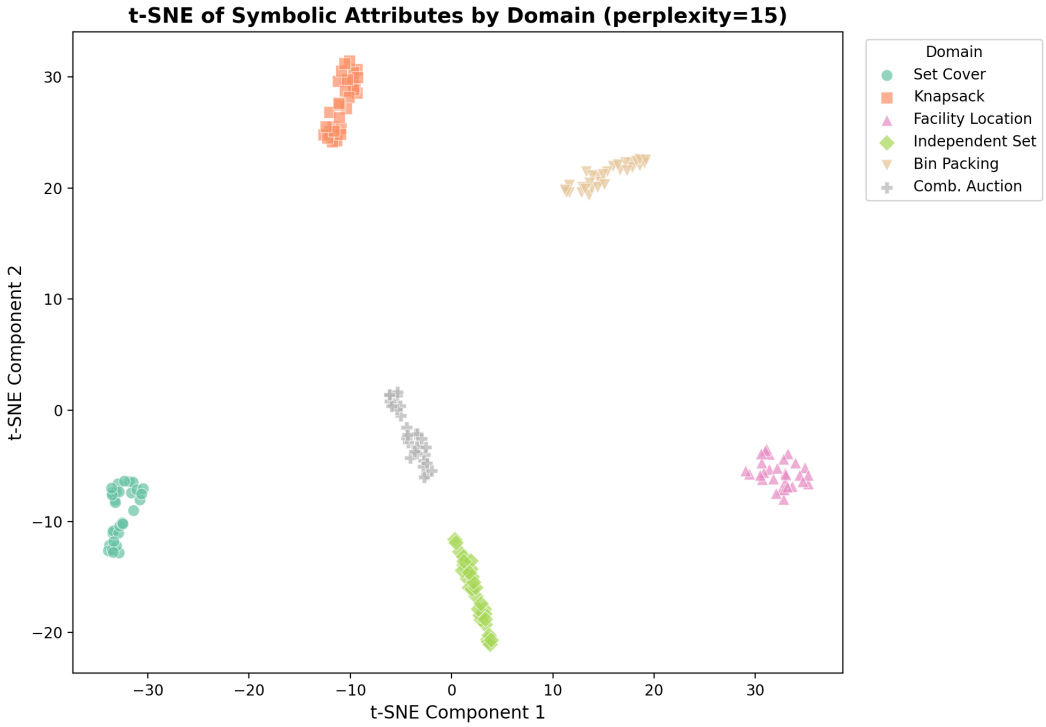

Figure 1: t-SNE of Symbolic Attributes by Domain (perplexity=15)

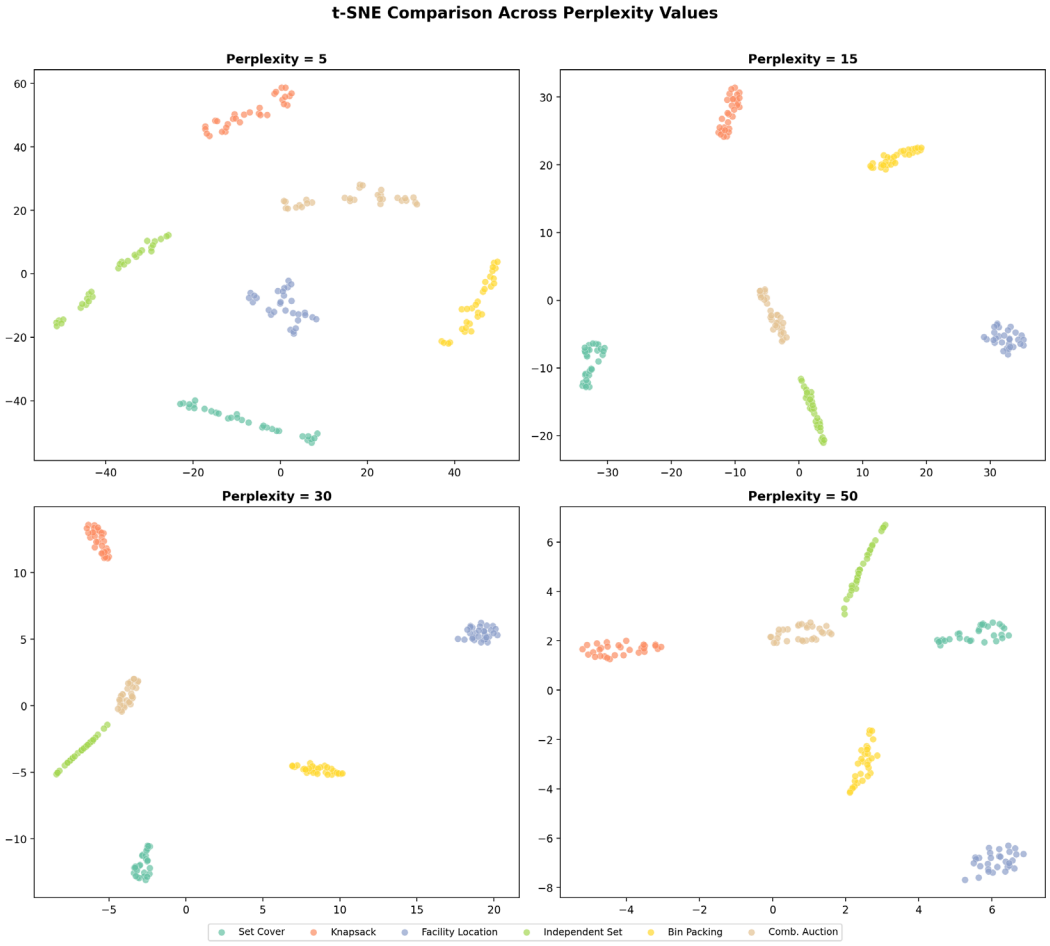

Figure 2: t-SNE Comparison Across Perplexity Values

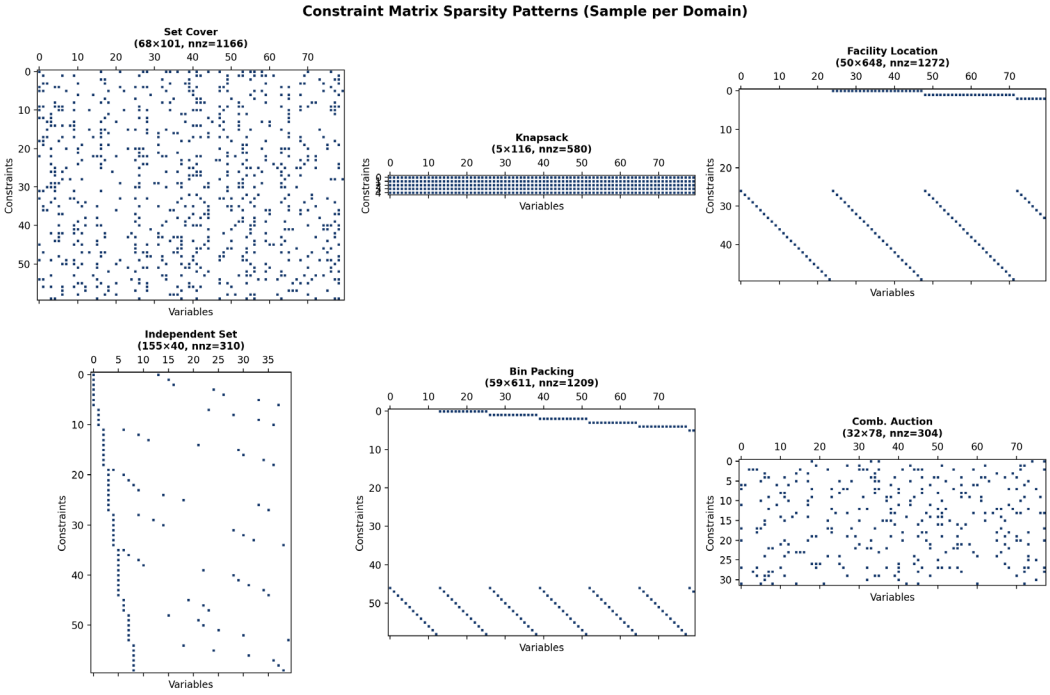

Figure 3: Constraint Matrix Sparsity Patterns

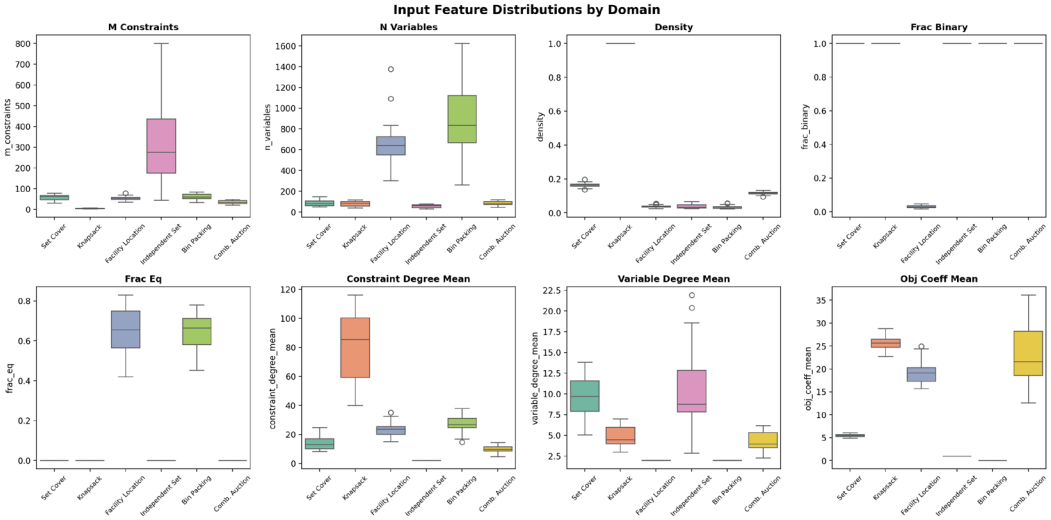

Figure 4: Input Feature Distributions by Domain

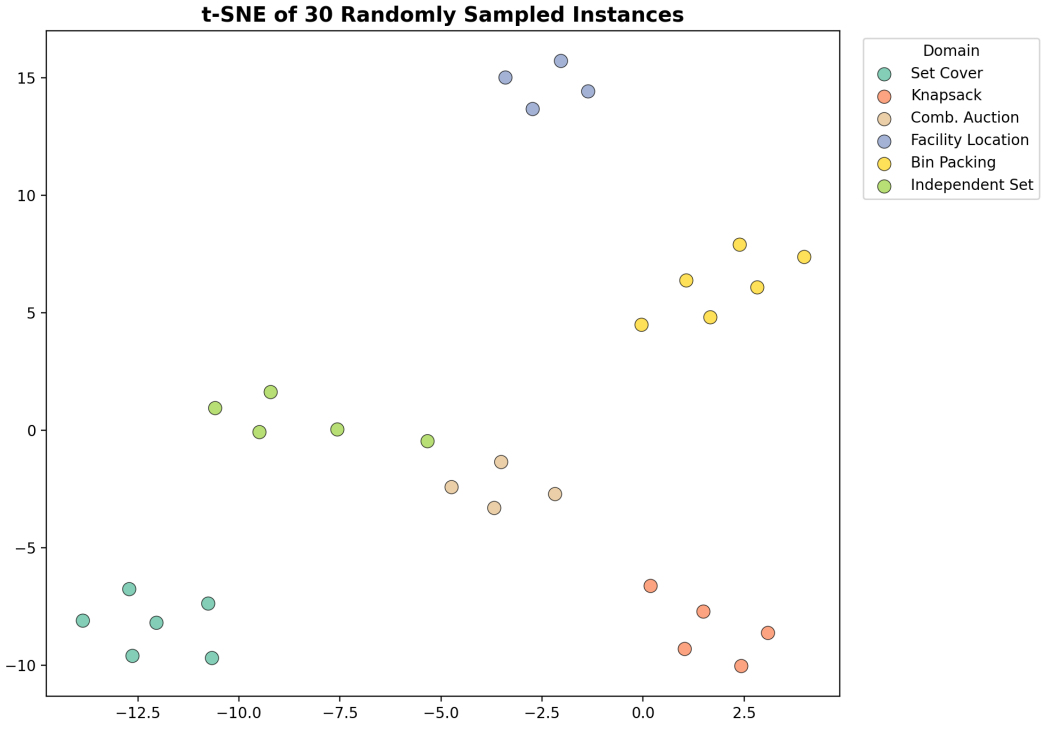

Figure 5: t-SNE of 30 Randomly Sampled Instances

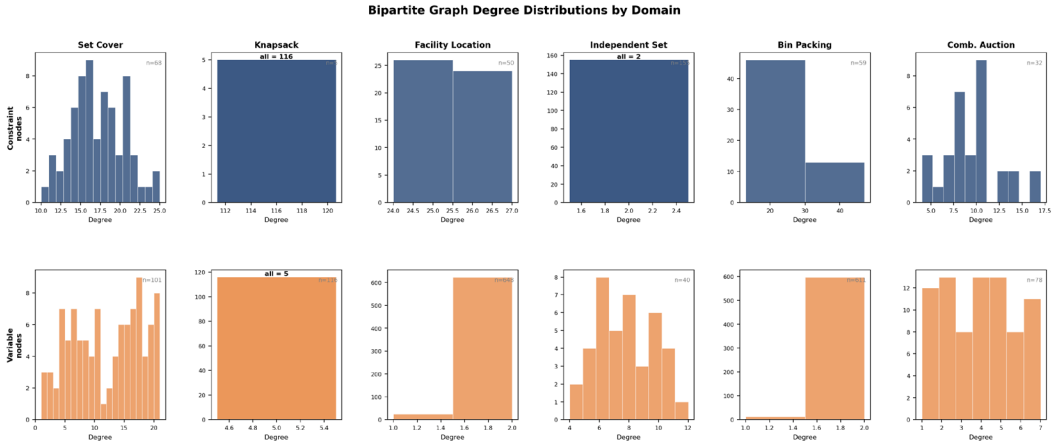

Figure 6: Bipartite Graph Degree Distributions

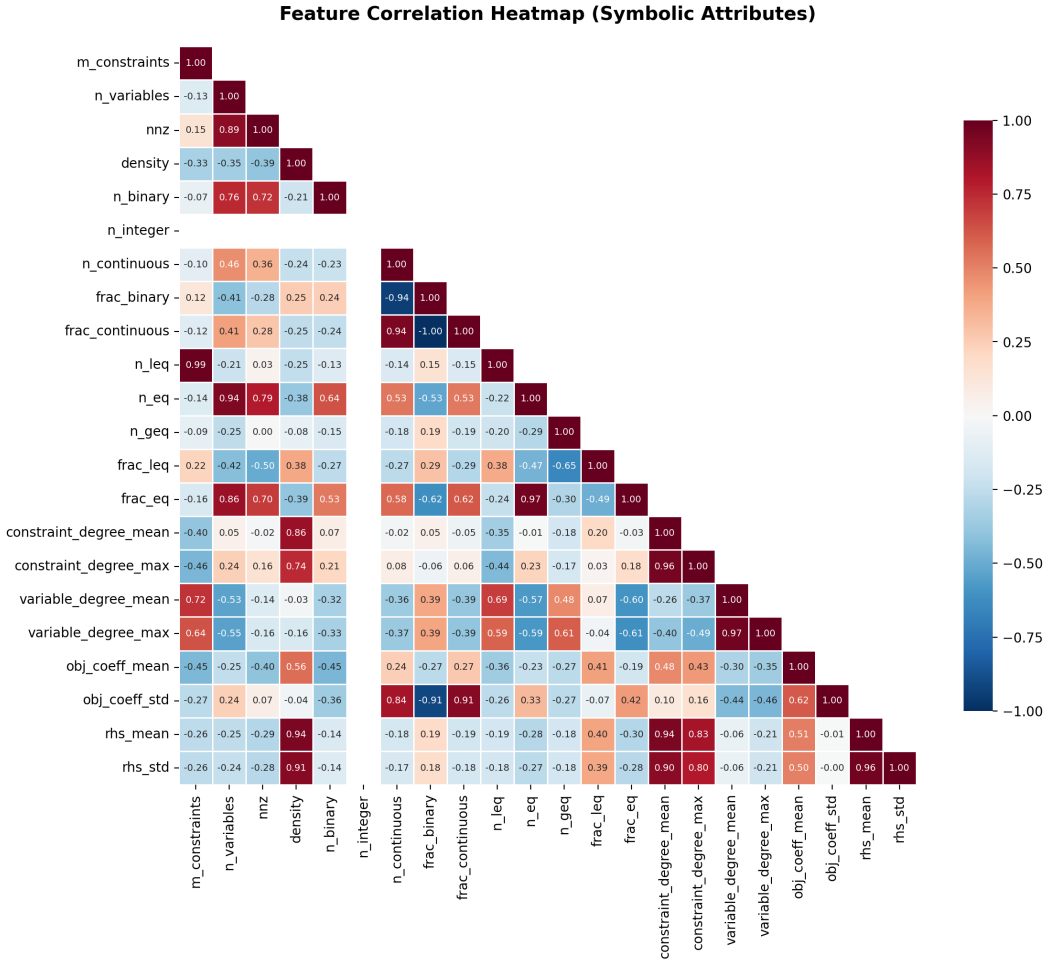

Figure 7: Feature Correlation Heatmap

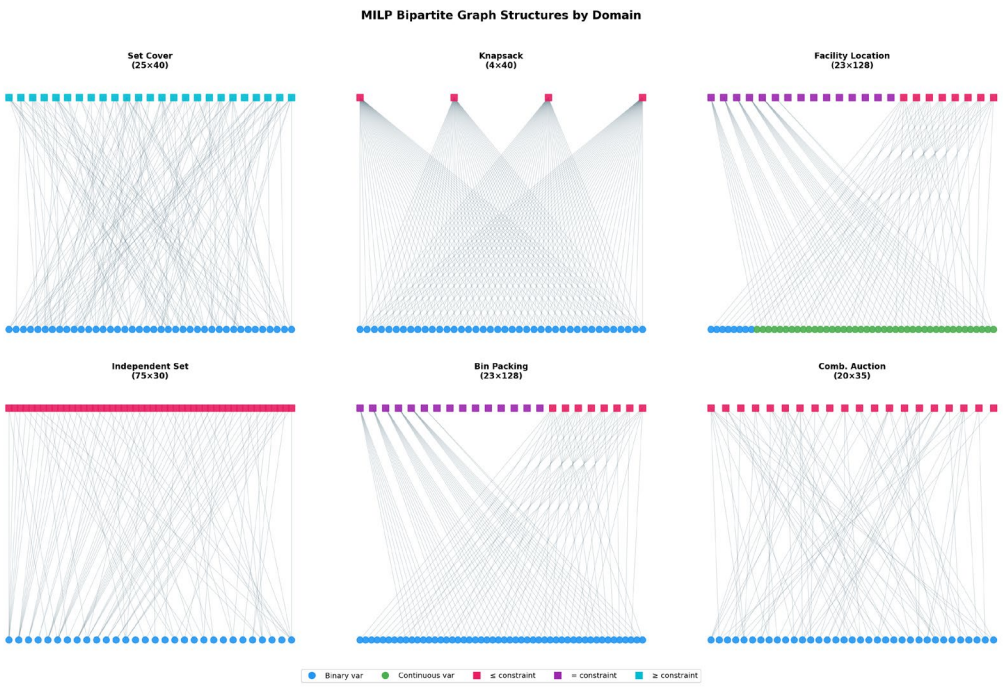

Figure 8: MILP Bipartite Graph Structures

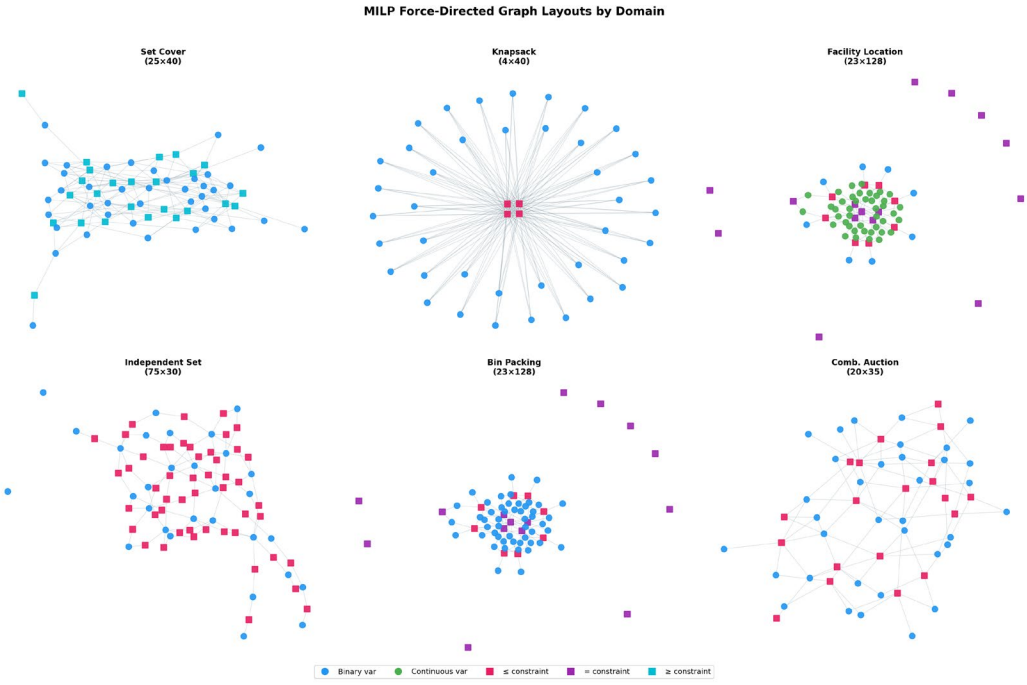

Figure 9: Force-Directed Graph Layouts

Part 4: Evaluation Metrics (20 pts)

4.1 Chosen Metrics & Rationale

I implemented five metrics:

1. Accuracy: To measure the fraction of correct predictions as a simple baseline.

2. AUROC: To measure how well the model distinguishes consistent vs inconsistent pairs.

3. F1 Score (macro): To evaluate the archetype classification, treating all classes equally.

4. Top-k Accuracy: For the diagnosis task, checking if the real error is in the top k predictions (tested for k=1, 3, 5).

5. Cosine Similarity Gap: To see if the embeddings for matching pairs are closer than non-matching pairs.

4.2 Alternative Metrics Considered

Plain accuracy was included as a baseline, but it is not enough when classes are imbalanced. That is why I also went with F1 and AUROC. I also considered precision/recall separately but F1 combines them which is easier.

4.3 Pros & Cons

Accuracy: Simple and interpretable, but misleading when classes are imbalanced.

AUROC: Good because it doesn't need a specific threshold, but can sometimes be misleading if the dataset is very imbalanced.

F1 (macro): Good for balancing classes, but gives equal weight to rare classes which might not always be desired.

Top-k Accuracy: Good for user-assistive tools, but k is somewhat arbitrary (tested k=1,3,5).

Cosine Similarity Gap: Good for checking the embeddings directly, but it doesn't always tell you how well the final classifier will work.

Part 5: Instruction Tuning / Prompt Design (15 pts)

5.1 Scenario 1 – Sentiment Classification

Review: "This place stinks, the service was awful and the food was not cooked. I will never come back here!"

Prompt:

Classify the sentiment of this review.
Review: "This place stinks, the service was awful and the food was not cooked. I will never come back here!"

Instructions:
- Reply with 'positive' or 'negative'.
- Only one word.

Sentiment:

Expected output: negative

Strategy: Constraining the output space forces the model to pick a valid class.

5.2 Scenario 2 – Emotion Recognition

Dataset: images of angry, sad, and happy faces.

Prompt:

Look at this face [IMAGE]. How is the person feeling?

Options: angry, sad, happy.
Respond with the emotion only.

Emotion:

Expected output: [one of the labels]

Strategy: Providing specific options prevents hallucinated or synonymous emotions.

5.3 Scenario 3 – Information Extraction

Paragraph: "The man, Edgar, flew to Italy to hike the Alps. He was looking forward to going skiing there."

Prompt 1:
Read this: "The man, Edgar, flew to Italy to hike the Alps. He was looking forward to going skiing there."
Who is the subject? Name only.
Answer: Edgar

Prompt 2:
Where is he going? Place only.
Answer: Italy

Prompt 3:
What will he do? Comma separated list.
Answer: hiking, skiing

Strategy: Breaking complex tasks into sub-prompts reduces ambiguity and confusion.

Part 6: Reflection (5 pts)

6.1 Most Interesting Topic

The t-SNE visualization was pretty cool. I was surprised that simple stats like density could separate the problems so well. Knapsack looks totally different from the others. It shows that we don't always need complex deep learning to see patterns.

6.2 Unexpected Challenge

I didn't expect setting up the environment to be so annoying. I spent a lot of time trying to get python-mip to work on Colab before realizing it needed system libraries I couldn't easily install. Writing the generators manually was a bit tedious but at least it worked.

6.3 Dataset Quality Assessment

The dataset is okay for a first assignment. 180 instances is small but enough to run the code. The main issue is that it's all synthetic, so it might be too 'perfect' compared to real world data. Also missing the text part is a big gap, but I'll add that later.

Bonus: Extra Credit Project (10 pts)

Task

Hateful Meme Detection. The goal is to classify memes as 'hateful' or 'not hateful'. This is a classic multimodal problem because the text usually isn't hateful on its own, and the image isn't hateful on its own. The hate comes from the combination of both.

Dataset

We used a subset of the Facebook Hateful Memes dataset (from HuggingFace). We ended up with about 650 training images and 162 validation images due to download constraints.

Modalities

We extracted three types of features:

1. Text (TF-IDF): A bag-of-words approach.
2. Image (Color Histograms): Simple RGB statistics.
3. CLIP Embeddings: Pre-trained Vision-Language Model embeddings (from 'openai/clip-vit-base-patch32').

Model

To keep the comparison fair and consistent, we used a Logistic Regression classifier for all experiments. We tested three configurations:
1. Text-only Baseline
2. Early Fusion (Text + Image Histograms)
3. CLIP Multimodal (Text + Image Embeddings)
We avoided complex neural networks to focus on the quality of the features themselves.

Evaluation Metrics

We used AUROC (Area Under ROC Curve) as the primary metric, which is standard for this dataset.

Results

Results on Validation Set:

Method	AUROC	Interpretation
1. Text-only (TF-IDF)	0.542	Barely above random guessing.
2. Simple Fusion (TF-IDF+RGB)	0.549	Adding color stats didn't help much.
3. CLIP Multimodal	0.601	Significant improvement.

Analysis:
The simple features failed because they don't capture the semantic relationship between text

and image. While the CLIP score (0.601) isn't perfect, it is visibly higher than the baselines, proving that pre-trained understanding is necessary for this task.

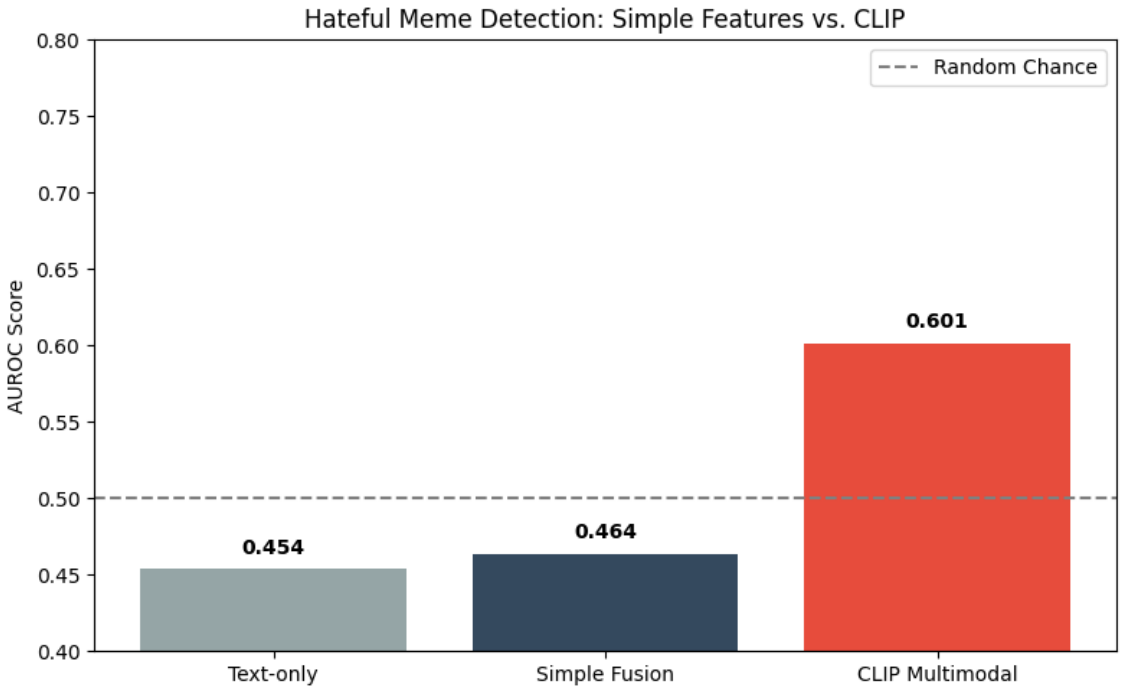

Figure B1: Hateful Meme Detection — Simple Feature Model Comparison vs CLIP (AUROC)

References

Liang, P. P., Zadeh, A., & Morency, L.-P. (2022). Foundations & Trends in Multimodal Machine Learning: Principles, Challenges, and Open Questions. arXiv:2209.03430.

Kiela, D., Firooz, H., Mohan, A., Goswami, V., Singh, A., Ringshia, P., & Testuggine, D. (2020). The Hateful Memes Challenge: Detecting Hate Speech in Multimodal Memes. NeurIPS 2020.

AI Use Disclosure

This assignment was completed with AI assistance (Claude, Anthropic). I used it to help write the MILP generator code since I was having trouble with the libraries, and also for checking some grammar in the report.